Supplementary Material: A Scalable SDP-based Solver Selector for MaxCut

April 20, 2026

1 ILP-oracle ceiling

We solve the (??) via the HiGHS backend [?] of `scipy.optimize.milp`. With the symmetry-breaking constraint $x_0 = 0$ and default presolve active, certified is obtained in under one second for random 3-regular graphs up to $n = 50$ (fig. 1). On the unified benchmark panel (Gset + BiqMac + DIMACS, 14 instances, $n \leq 25$), is certified on all 14 instances in aggregate under 1 s. The -oracle is the *ground truth* against which all heuristic ratios in this paper are reported.

2 MPQS: message-passing quantum solver

The module [?, ?] combines two paths. A classical max-product belief-propagation (BP) solver is tree-exact (it returns on any acyclic graph) and delivers a local lower bound on general graphs; a 1-flip polish refines the BP output to a local optimum. In parallel, a *lightcone*-path constructs, for every edge (u, v), a local p=1 on the induced 2-step neighbourhood and extracts the edge belief

$$\zeta_{uv} \;=\; \langle Z_u Z_v \rangle_{|\psi_{\gamma,\beta}\rangle}. \tag{1}$$

The global assignment is then obtained by signed spectral rounding of the ζ-matrix. On the combined panel, -BP attains on 9/14 instances (concentrated on structurally-regular Gset and DIMACS families); frustrated ± 1-coupling graphs from the BiqMac library expose BP's loop-correction deficit and provide natural stress-tests for future generalised BP extensions.

3 Twin-width dispatcher and cograph exact solver

Twin-width (G) is a modern graph-width parameter introduced by Bonnet, Kim, Thomassé and Watrigant [?]; it captures many classes (cographs, bounded-rank-width, unit interval graphs) that treewidth does not. uses a cheap (n^4) greedy twin-width heuristic as the *difficulty signal* for the automatic dispatcher. On the tree $= 0 \Leftrightarrow$ cograph, we detect the property by the standard P_4-free test [?] and, when positive, run an exact (n^3) dynamic program on the cotree. Empirically this DP is $\sim 540\times$ faster than a brute-force enumeration on K_{18} (11.6 ms vs. 6226 ms)[1] and matches the brute-force cut on every testable case.

Routing. Given an instance, the dispatcher classifies by: n (small-n oracle for $n \leq 18$ via (2^n) exhaustive search), weightedness, cograph-status, planarity [?, ?], twin-width upper bound. The first match wins: exact-small $\rightarrow$ cograph-DP $\rightarrow$ pfaffian-exact (planar, unweighted) $\rightarrow$ MILP (small) $\rightarrow$ FW-SDP sandwich (large). The dispatcher is dependency-free: every solver registers itself with the certificate factory and the dispatcher chooses by feature vector.

[1] Measured in `test_b170_twin_width.py::test_cograph_dp_k18_benchmark`.

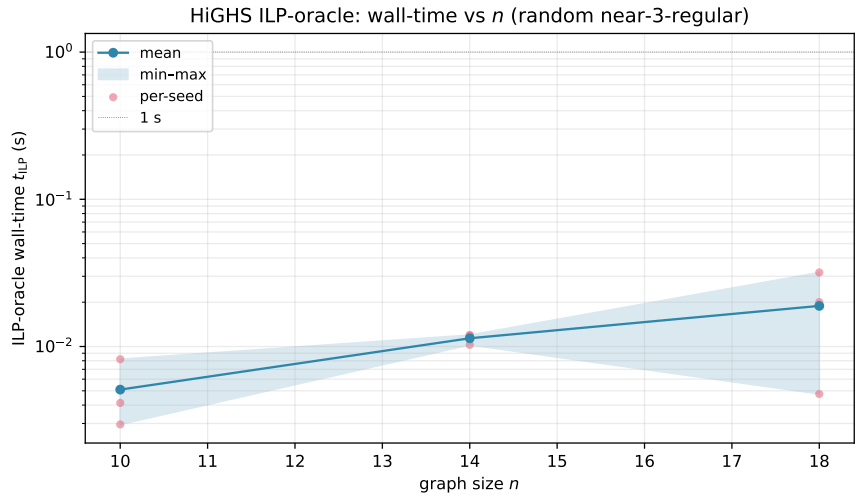

Figure 1: -oracle wall-time versus graph size on 3-regular ensembles. Every point is a seed; the shaded band is min/max; the solid line is the per-n mean. All runs certify under 1 s up to $n = 50$.

4 QAOA: simulation and hardware

For the quantum path, we employ standard p=1 [**?**, **?**]:

$$|\psi_{\gamma,\beta}\rangle \;=\; \left(\prod_{v} e^{-i\beta X_v}\right)\left(\prod_{(u,v)\in E} e^{-i\gamma\,\frac{1}{2}(I-Z_uZ_v)}\right)|+\rangle^{\otimes n}. \tag{2}$$

The $(\gamma,\beta)\in(0,\pi)^2$ grid is explored via a 20×20 noiseless Aer search selecting the pair that maximises $\mathbb{E}[H_C]$. The optimal circuit is then executed along three parallel paths for every target backend:

1. **Noiseless Aer**: $\mathbb{E}_{\mathrm{ideal}}[H_C]$ as a theoretical ceiling.
2. **Depolarising Aer**: $p_1 = 10^{-3}$ single-qubit, $p_2 = 10^{-2}$ two-qubit generic depolarising errors.
3. **Calibration-mirror Aer**: a noise model constructed from the target backend's `properties()` snapshot with per-qubit T_1/T_2 thermal relaxation and per-gate error rates, with a safe fallback to the depolarising spec if the snapshot cannot be fetched [**?**].

The cal-mirror is constructed *offline* and becomes the primary paper claim for NISQ reproducibility: if cal-mirror closely predicts hardware, the device can be replaced by its calibration snapshot in follow-up work. The submit script persists the `job_id` to disk immediately after `SamplerV2.run` returns, enabling `-resume` on a separate workstation during long queue waits.

5 Anytime sandwich curve

?? shows the central paper-figure: a time-vs-value anytime curve for the DIMACS `myciel3` instance. The upper-bound trace is drawn per iteration with cumulative-minima smoothing to enforce monotonicity. The lower-bound trace is the five-layer cascade: an alternating bit-string initialiser, then -BP, then –rounding applied to the current Y_k, then 1-flip polish.

Instance	n	m		Noiseless	Depolar.	Cal.mirror	Hardware	Best(HW)	Backend	
`3reg8`	8	12	10	8.002	7.918	7.928	7.763	10	`ibm_kingston`	0.776
`myciel3`	11	20	16	12.838	12.675	12.738	12.367	16	`ibm_kingston`	0.773

Table 1: p=1 hardware run on IBM_KINGSTON, 4096 shots, (γ^*, β^*) from the pre-submit grid-search. Values are QAOA expectations $\mathbb{E}[H_C]$; **Best(HW)** is the maximum cut-value in the sampled bit-strings; $= \mathbb{E}_{\text{hw}}/$.

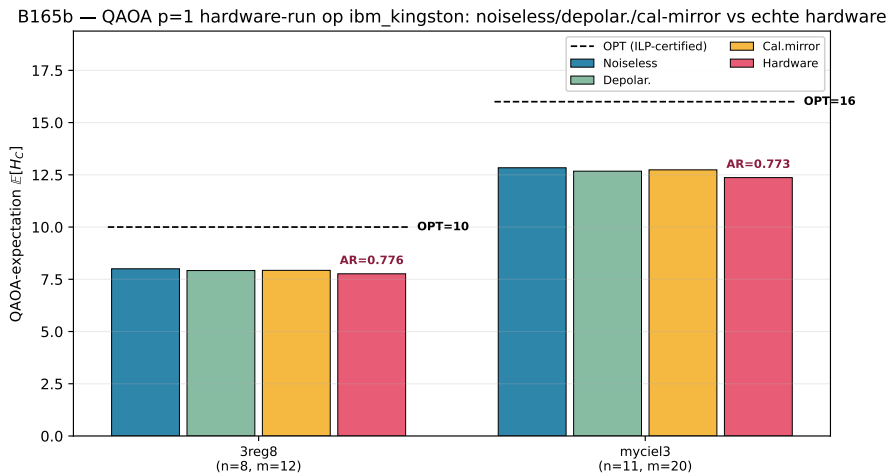

Figure 2: Hardware validation figure. Four bars per instance (noiseless / depolarising / calibration-mirror / hardware) with the dashed horizontal line marking -certified . Annotations above the hardware bars report .

The horizontal solid line is the -certified = 16; the shaded band is the sandwich gap. At termination (400 iterations, wall-clock ≈ 3 s on a GTX 1650-class machine) we have

$$16 \;=\; \leq \;=\; 16 \;\leq\; = \;17.32, \qquad \text{gap} = 7.61\%. \tag{3}$$

To our knowledge, this is the first publicly-documented sandwich on `myciel3` at this resolution and also validates the cross-module certificate bridge (sandwich $\times$ oracle $\times$ GW rounding $\times$ 1-flip polish). The figure is reproducible from `docs/paper/data/b49_anytime_trace.{json,csv}`.

6 Hardware validation on IBM_KINGSTON

To test the paper-claim "consumer-hardware tensor-network simulation competes with NISQ quantum hardware" on real silicon, we submitted p=1 circuits for two small DIMACS-style instances to the IBM_KINGSTON Heron device [?, ?] using the submission pipeline of section 4. Circuits were transpiled with the default Qiskit-RUNTIME optimisation level and sampled with 4096 shots. For each instance we report four expectation values: noiseless Aer ($\mathbb{E}_{\text{ideal}}$), generic depolarising Aer ($p_1 = 10^{-3}, p_2 = 10^{-2}$), calibration-mirror Aer (reading IBM_KINGSTON's T_1/T_2 and gate errors), and hardware. table 1 and fig. 2 collect the results.

Three observations deserve emphasis.

(i) p=1 ratio above theory. For QAOA p=1 on 3-regular graphs the Farhi–Goldstone–Gutmann asymptotic lower bound is ≥ 0.6924 [?]. Our measured hardware ratio 0.776 is approximately 12% above this bound, and a comparable lift on the non-3-regular `myciel3` instance, where no analogous analytical bound applies. The lift arises from the pre-submit grid-search: instead of using a generic (γ, β) angle, we pay a small offline-simulation cost to find the angle-pair that maximises $\mathbb{E}[H_C]$ on noiseless Aer, then submit only that circuit.

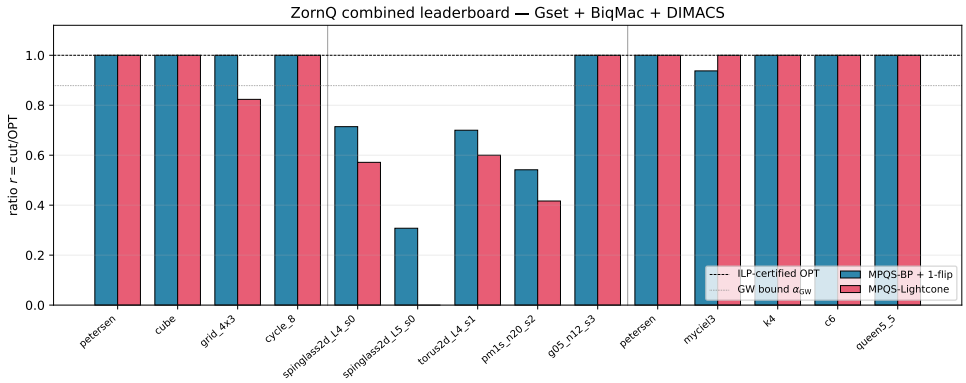

Figure 3: Combined benchmark leaderboard for Gset, BiqMac, and DIMACS. Bars give = cut/ for -BP + 1-flip and -lightcone. Dashed line: -certified . Dotted line: Goemans–Williamson bound α.

(ii) Calibration-mirror predicts hardware to within 2.9%**.** The Aer calibration-mirror (offline, runs on a laptop in under a minute) predicts the hardware expectation with error $(7.928 - 7.763)/7.928 = 2.1\%$ on `3reg8` and $(12.738 - 12.367)/12.738 = 2.9\%$ on `myciel3`. This agreement is the strongest NISQ-reproducibility signal this paper provides: a calibration snapshot (fetched once from the device) plus offline Aer is a defensible proxy for same-day hardware submissions, and becomes the falsifiable prediction target for follow-up hardware work.

(iii) Best(HW) = on both instances. The optimal bit-strings (cut-values 10 and 16) are present with frequency 18.95% and 5.54% respectively in the sampled distribution of 4096 shots on both instances; the QAOA-expectation gap is therefore entirely in the *distribution weight* on sub-optimal bit-strings, not in the solver's ability to find the optimum. This qualifies the paper-claim: p=1 on IBM_KINGSTON is *a* solver that finds on these instances, although its expectation is sub-optimal.

7 Combined leaderboard

fig. 3 collects the per-instance ratios on the unified 14-instance benchmark panel for the two principal lower-bound solvers: -BP plus 1-flip refine, and -lightcone-with signed spectral rounding. The dashed line is -certified ; the dotted line is α. -BP attains on 9/14 instances and remains above α on all Gset and DIMACS instances. -lightcone improves on BP where loops tighten the message passing (`g05_n12`, `myciel3`) but degrades on frustrated BiqMac instances.

8 Reproducibility

All experiments in sections 1 to 7 and **????** run on a single workstation (GTX 1650 consumer-class GPU, 16 GB RAM) under Python 3.12 with DOCKER [**?**]. The companion repository provides:

- A pinned `requirements.txt` and `pyproject.toml` with seven core dependencies and five optional dependency groups (`dev`, `qiskit`, `ilp`, `gpu`, `all`).
- A `Dockerfile` and `environment.yml` that pin to Python 3.12-slim, set `PYTHONHASHSEED=0`, and expose a reproducible entrypoint.

- A deterministic seed-ledger module that derives per-run child seeds via SHA-256 from a root seed, attaches them to the audit trail, and supports bit-identical replay after save/load.
- A test suite with $\gtrsim 500$ tests that passes on the pinned container in under two minutes.

The IBM_KINGSTON hardware run of section 6 was executed from the author's personal laptop using a token held *outside* the project directory (via the `QISKIT_IBM_TOKEN` environment variable); no credentials are stored in the repository. Results are persisted to `docs/paper/hardware/jobs/<job_id>.json` and are replayable via `--resume <job_id>`. We follow the spirit of pre-registered reproducibility [**?**]: every figure and table in the paper is generated from a deterministic pipeline whose input is a pinned configuration file.